

Abhishek Raghav

Btech CSE

CU24250156

✓ Experiment No. 6

Experiment Name

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Aim

To implement and evaluate different image segmentation techniques for dividing a digital image into meaningful regions for object localization, scene understanding, and other computer vision tasks.

Purpose

Image segmentation divides an image into multiple meaningful regions or objects based on pixel intensity, color, texture, or boundaries. It helps isolate regions of interest and is useful in object detection, medical diagnosis, autonomous navigation, and industrial inspection.

✓ 1. Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

✓ 2. Load the Image

```
# Load image
image = cv2.imread("/content/positive_quotes_wallpaper_hd_4k_by_sahibdm_ddc2cya-pre.jpg")

if image is None:
    print("Image not found")
else:
    print("Image loaded successfully")

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display original image
plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image loaded successfully

Original Image



Explanation

This experiment specifies converting the image to grayscale and using Gaussian Blur for basic noise reduction before segmentation.

✓ 3. Convert Image to Grayscale and Apply Gaussian Blur

```
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian Blur
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Display
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.show()
```

Grayscale Image



Gaussian Blurred Image



4. Global Thresholding

Global thresholding uses a fixed threshold value to separate foreground and background.

```
# Global Thresholding
threshold_value = 127

ret, global_thresh = cv2.threshold(
    blur,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 6))
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```

Global Thresholding



Explanation

If the pixel intensity is greater than the selected threshold, it is assigned to the foreground; otherwise, it is assigned to the background.

The experiment uses global thresholding to separate foreground objects from the background.

✓ 5. Otsu's Thresholding

Otsu's method automatically determines an appropriate threshold from the image histogram.

```
# Otsu's Thresholding
otsu_value, otsu_thresh = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Optimal Otsu Threshold:", otsu_value)

plt.figure(figsize=(8, 6))
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

Optimal Otsu Threshold: 42.0

Otsu's Thresholding



Explanation

Unlike global thresholding, where we manually select a threshold, Otsu's method automatically determines the threshold that best separates the image into two intensity classes.

The experiment specifically requires comparing Otsu's automatically selected threshold with global thresholding.

✓ 6. Adaptive Thresholding

Adaptive thresholding is useful when different parts of an image have different illumination levels.

```

# Adaptive Mean Thresholding
adaptive_mean = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# Adaptive Gaussian Thresholding
adaptive_gaussian = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

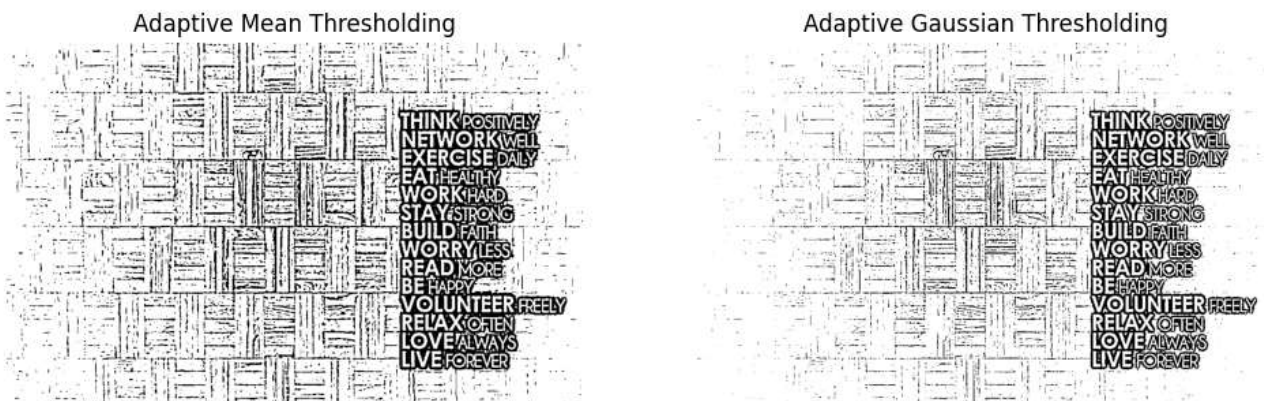
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(adaptive_mean, cmap="gray")
plt.title("Adaptive Mean Thresholding")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(adaptive_gaussian, cmap="gray")
plt.title("Adaptive Gaussian Thresholding")
plt.axis("off")

plt.show()

```



Explanation

Adaptive thresholding calculates a different threshold for different regions of the image. Therefore, it performs better than simple global thresholding when illumination is uneven.

The uploaded experiment specifically identifies adaptive thresholding for uneven illumination conditions.

✓ 7. Watershed Segmentation

Watershed segmentation is particularly useful for separating objects that are touching or overlapping.

```

# Binary image using Otsu
_, binary = cv2.threshold(
    blur,

```

```

    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove noise using morphological opening
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Find sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Find sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.7 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labeling
num_labels, markers = cv2.connectedComponents(sure_fg)

# Add one to all labels
markers = markers + 1

# Mark unknown region as 0
markers[unknown == 255] = 0

# Apply watershed
image_watershed = image.copy()

markers = cv2.watershed(
    image_watershed,
    markers
)

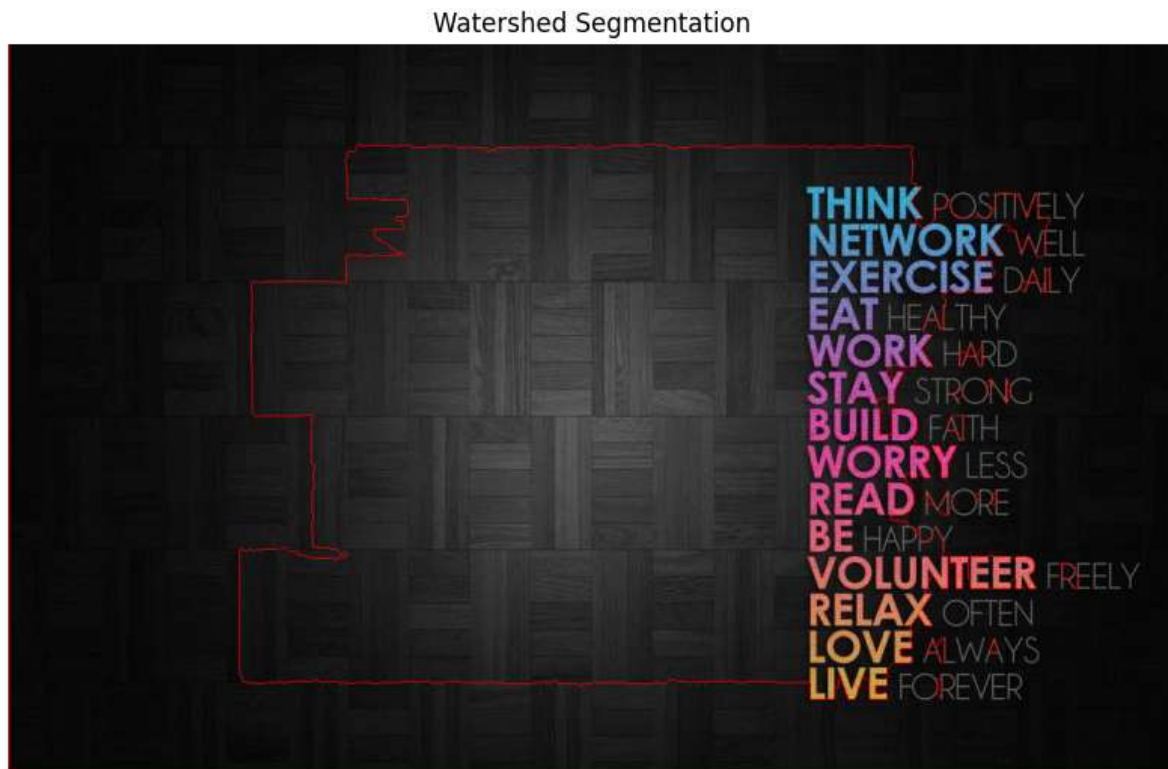
# Mark boundaries in red
image_watershed[markers == -1] = [0, 0, 255]

# Convert for display
watershed_rgb = cv2.cvtColor(
    image_watershed,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(10, 7))

```

```
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```



Explanation

The Watershed algorithm treats the image like a topographic surface. Different regions are separated using boundaries called watershed lines.

It is especially useful when objects are touching or overlapping, because it can separate individual objects based on their distance and connected regions.

✓ 8. K-Means Clustering

K-Means is a clustering-based segmentation technique that groups pixels with similar colors into different clusters. The experiment specifies using K-Means for color-based image segmentation.

```
# Convert image to RGB
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Reshape image into a list of pixels
pixels = rgb_image.reshape((-1, 3))

# Convert to float
pixels = np.float32(pixels)

# Number of clusters
k = 3

# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
```

```

)

labels = kmeans.fit_predict(pixels)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Replace pixels with cluster centers
segmented_pixels = centers[labels]

# Reshape back to image
segmented_image = segmented_pixels.reshape(
    rgb_image.shape
)

# Display
plt.figure(figsize=(8, 6))
plt.imshow(segmented_image)
plt.title("K-Means Image Segmentation")
plt.axis("off")
plt.show()

```

K-Means Image Segmentation



Explanation

K-Means divides pixels into K clusters based on similarity.

For example:

Original Image ↓ Pixel Values ↓ K-Means Clustering ↓ Cluster 1 Cluster 2 Cluster 3 ↓ Segmented Image

✓ 9. Display All Segmentation Results

◆ Gemini

```

plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)

```

```
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Threshold")
plt.axis("off")

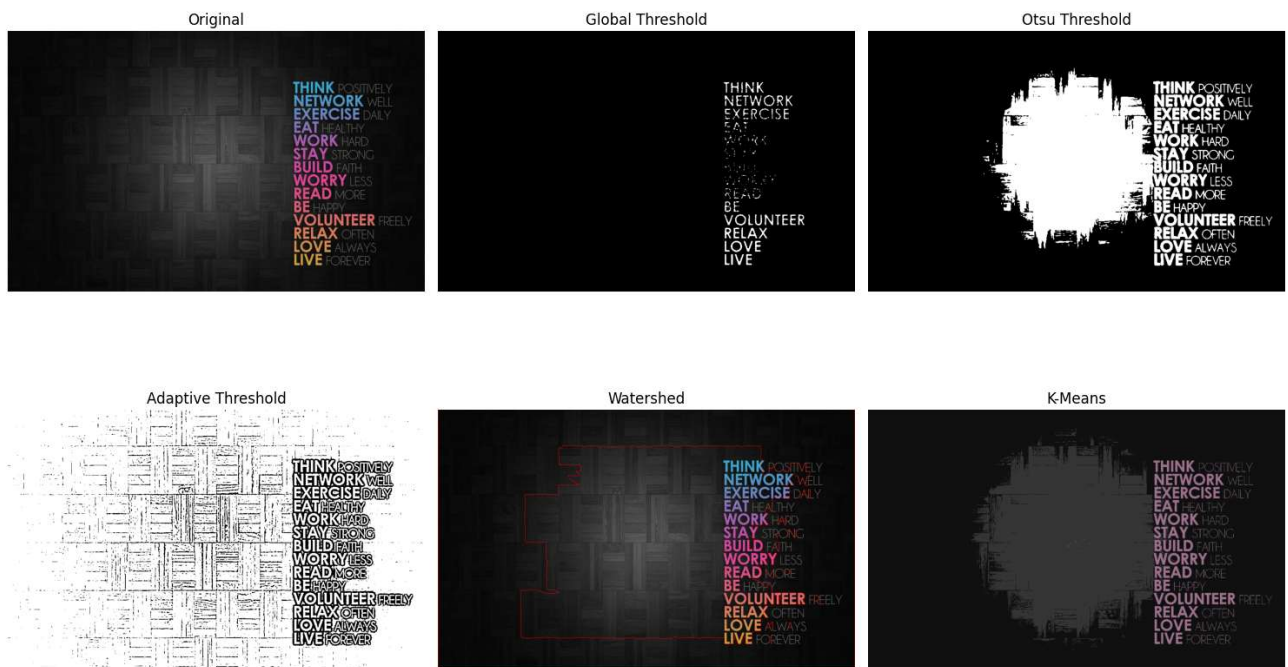
plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Threshold")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_gaussian, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_rgb)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



10. Comparison of Segmentation Techniques

Technique	Basic Principle	Advantages	Limitations
Global Thresholding	Fixed threshold	Simple and fast	Poor with uneven illumination
Otsu's Thresholding	Automatic threshold selection	No manual threshold required	Mainly suitable for two-class separation
Adaptive Thresholding	Local threshold	Works with uneven lighting	More computationally expensive
Watershed	Region/boundary based	Separates touching objects	Can cause over-segmentation
K-Means	Pixel clustering	Good for color segmentation	Need to select number of clusters

The experiment asks these methods to be compared based on object boundaries, accuracy, and computational efficiency.

Questions and Answers

Q1. What is image segmentation? Why is it a fundamental step in computer vision?

Answer:

Image segmentation is the process of dividing an image into multiple meaningful regions or objects based on properties such as pixel intensity, color, texture, or boundaries.

It is fundamental because it helps isolate important regions of an image. This makes further tasks such as object detection, recognition, medical diagnosis, autonomous navigation, and industrial inspection easier and more accurate.

Q2. Differentiate between Image Segmentation and Image Classification.

Image Segmentation	Image Classification
Divides an image into regions	Assigns a class/label to an image or object
Works at pixel/region level	Usually works at image/object level
Identifies boundaries	Identifies category
Produces a segmented image or mask	Produces class labels
Example: separating tumor from healthy tissue	Example: classifying an image as tumor/no tumor

Example

If an image contains a car:

Classification: "This is a car."

Segmentation: "These exact pixels belong to the car."

Q3. Explain Global, Otsu's and Adaptive Thresholding.

Global Thresholding

A single threshold value is applied to the complete image.

$\text{Pixel} > T \rightarrow \text{White}$ $\text{Pixel} \leq T \rightarrow \text{Black}$

It is simple and fast but may fail when illumination varies.

Otsu's Thresholding

Otsu's method automatically determines an optimal threshold by analyzing the intensity distribution of the image.

It is useful when the image can be reasonably separated into foreground and background.

Adaptive Thresholding

Adaptive thresholding calculates thresholds for different local regions instead of using one threshold for the entire image.

It is useful for images with uneven illumination.

Q4. What is the Watershed Algorithm? Why is it useful for overlapping objects?

Answer:

The Watershed algorithm is an image segmentation method that considers an image as a topographical surface.

It divides the image into different regions using boundaries called watershed lines.

It is useful for separating overlapping or touching objects because it can identify different object regions using foreground markers and distance information.

For example, if two cells touch each other in a medical image, Watershed can help separate them into two individual objects.

The experiment specifically requires Watershed to separate overlapping or touching objects.

Q5. How is K-Means Clustering applied to image segmentation?

Answer:

K-Means treats every pixel as a data point.

The steps are:

1. Select the number of clusters K.
2. Convert image pixels into numerical feature vectors.
3. Initialize K cluster centers.
4. Assign each pixel to the nearest cluster.
5. Recalculate cluster centers.
6. Repeat until the clusters become stable.
7. Replace each pixel with its cluster center.
8. Reconstruct the segmented image.

For color images, RGB values can be used as the features.

Q6. Compare threshold-based and clustering-based segmentation.

Threshold-Based	Clustering-Based
Uses threshold values	Uses similarity between pixels
Usually simpler	More computationally demanding
Good for simple foreground/background	Can handle multiple regions
Mainly based on intensity	Can use color/features
Example: Otsu	Example: K-Means

Threshold methods are generally simpler, whereas clustering methods can provide more flexible segmentation of images with multiple regions.

Q7. What challenges occur with complex backgrounds or varying illumination?

Answer:

Major challenges include:

- Similar colors between foreground and background.
- Uneven illumination.
- Shadows.
- Noise.
- Complex textures.
- Low contrast.
- Overlapping objects.
- Gradual boundaries between objects.

For example, global thresholding may fail when one part of an image is bright while another part is dark. Adaptive thresholding can be more suitable in such situations.

Q8. Mention five real-world applications of image segmentation.

Answer:

Five important applications are:

- Medical image analysis – separating tumors or organs.
- Autonomous vehicles – identifying roads, vehicles, and pedestrians.
- Satellite image analysis – identifying land, water, and buildings.
- Industrial inspection – detecting defects in manufactured products.
- Object detection and recognition – separating objects from backgrounds.

The experiment specifically identifies medical diagnosis, autonomous navigation, industrial inspection, and object detection among important applications.

Q9. How does image segmentation improve object detection and recognition?

Answer:

Segmentation separates the important object from the background.

This allows the system to:

- Focus on the region of interest.
- Reduce unnecessary background information.
- Improve object boundaries.
- Extract useful features more accurately.
- Improve recognition accuracy.
- Make object localization easier.

Therefore:

Original Image ↓ Image Segmentation ↓ Region of Interest ↓ Feature Extraction ↓ Object Detection / Recognition

Segmentation helps isolate regions of interest for subsequent computer vision tasks.

Q10. Compare traditional segmentation with U-Net and Mask R-CNN.

Traditional Techniques	Deep Learning Techniques
Use manually designed rules	Learn features automatically
Examples: Thresholding, Watershed, K-Means	Examples: U-Net, Mask R-CNN
Usually require less training data	Generally require training data
Simple and easier to implement	More complex
Lower computational requirements in simple cases	Usually require more computational resources
Good for simple images	Better suited to complex segmentation tasks
Performance depends heavily on image conditions	Can learn complex patterns from data

U-Net

U-Net is a deep learning architecture commonly used for semantic segmentation, particularly in biomedical image segmentation.

Mask R-CNN

Mask R-CNN performs object detection along with generating a segmentation mask for each detected object.

Thus, traditional methods are useful for relatively simple segmentation problems, while deep learning approaches can handle more complex image patterns when sufficient training data is available.

Result

The different image segmentation techniques—Global Thresholding, Otsu's Thresholding, Adaptive Thresholding,